

Projet “Soirée Plateau” — Version solo

Plateforme web de gestion de soirées de jeux de société entre amis. Projet pédagogique d'**une semaine en autonomie** — formation **Concepteur Développeur d'Applications (CDA)**.

1. Contexte & enjeu

Une bande de joueurs de jeux de société organise régulièrement des soirées entre amis. Aujourd'hui, l'organisation se fait à coup de Doodle pour les dates, de groupe WhatsApp pour discuter, et d'un Google Sheet pour tenir l'inventaire des jeux et les notes.

Le résultat : des participants oubliés, des jeux apportés en double, des soirées où personne ne sait à quoi jouer, et zéro mémoire collective sur ce qui a plu ou non.

Mission : concevoir et développer **seul(e)** une application web qui centralise tout cela — création de soirées, RSVP des invités, sélection des jeux par l'hôte, et notation des jeux après coup.

L'objectif n'est **pas** de produire une application commercialisable. C'est de **traverser tout le cycle de vie d'un projet logiciel** en une semaine, de la conception au déploiement simulé. Le périmètre fonctionnel est volontairement resserré pour que vous puissiez passer du temps sur la qualité, les tests, l'industrialisation et la documentation — qui sont au moins aussi évalués que les fonctionnalités.

Règle d'or : il vaut bien mieux livrer un **MVP fini, testé, documenté et dockerisé** qu'un grand périmètre cassé. Le brief distingue clairement ce qui est indispensable de ce qui est facultatif — résistez à la tentation de tout traiter.

2. Objectifs pédagogiques

Ce projet vise à mobiliser, dans un cadre intégré, les compétences clés du REAC CDA :

Activité-type REAC	Compétences mobilisées
AT1 — Développer la partie front-end	Maquettes, intégration responsive, consommation API
AT2 — Développer la partie back-end	Architecture, API REST, persistance, sécurité, tests
AT3 — Concevoir et développer une application multicouche	Modélisation des données , UML, séparation des couches
AT4 — Préparer le déploiement	Dockerisation, CI/CD, documentation, simulation de déploiement

Focus particulier sur la **conception de la base de données**. Le brief vous décrit le métier, les règles, les contraintes — c'est à vous de **modéliser le schéma**. Aucun modèle de données ne vous est fourni : les choix d'entités, d'attributs, de relations, de cardinalités et de contraintes d'intégrité font partie intégrante de l'évaluation.

3. Périmètre fonctionnel

3.1 Ce qui est dans le scope (MVP)

- Inscription, connexion
- Catalogue de jeux (consultation pour tous, gestion réservée aux admins)
- Création de soirées
- RSVP des invités (oui / non / peut-être)
- Sélection des jeux apportés par l'hôte (parmi le catalogue)
- Notation (1 à 5) des jeux **après** la soirée par les participants
- Deux rôles : **Membre** et **Admin**

Les **extensions** (gestion de profil, annulation de soirée, historique, modération admin) sont décrites en section 5 mais **ne sont pas obligatoires** : à traiter seulement si le MVP est solidement livré.

3.2 Ce qui n'est PAS dans le scope

Pour tenir le périmètre solo, ces sujets sont **explicitement exclus** :

- Notifications email ou push
- Système de vote pour les jeux (l'hôte décide)
- Messagerie, commentaires, fil d'actualité
- Recommandations algorithmiques
- Import depuis BoardGameGeek ou autre API tierce
- Internationalisation
- Application mobile native
- Gestion fine des permissions au-delà des deux rôles
- Upload d'images avec stockage objet (utiliser des URLs en saisie texte)

Scope creep alert : si une fonctionnalité n'est pas listée dans la section 3.1 ou en extension dans la section 5, elle n'a pas à être livrée. Notez vos idées dans un fichier **BACKLOG.md** et passez à la suite.

4. Glossaire métier

Ces termes sont utilisés tout au long du brief. Ils décrivent le **vocabulaire du domaine**, pas la structure technique : à vous de traduire ces concepts en modèle de données.

- **Utilisateur** : personne inscrite sur la plateforme. Possède un rôle (membre ou admin).
- **Jeu** : entrée du catalogue de jeux de société, avec ses caractéristiques (nombre de joueurs, durée, complexité...).
- **Soirée** : évènement organisé par un utilisateur (l'hôte) à une date et un lieu donnés, avec une capacité maximum et une liste de jeux apportés.
- **Hôte** : utilisateur qui organise une soirée. Une soirée a un seul hôte.
- **RSVP / Participation** : réponse d'un utilisateur à une soirée (oui / non / peut-être).
- **Participant confirmé** : utilisateur ayant répondu "oui" à une soirée.
- **Note** : appréciation (1 à 5) d'un jeu donné, dans le contexte d'une soirée passée précise, donnée par un participant confirmé.

5. User stories (MVP, Extensions, Modération)

Les user stories sont classées par **priorité** :

Priorité	Quand l'aborder	Conséquence si non livrée
MVP	Indispensable. À livrer avant tout autre développement .	Projet considéré comme incomplet
Extension	Si le MVP est solide ET testé ET dockerisé.	Aucune, juste mes félicitations
Modération admin	Si tout le reste est livré.	Aucune. Pur bonus.

Stratégie recommandée : verrouillez **toutes les US MVP** avant d'en tenter une seule en Extension. Une US d'extension à moitié faite vaut moins qu'une US MVP absente : elle pollue le périmètre, complique les tests et risque de casser le MVP.

MVP — Indispensable

Les neuf user stories ci-dessous constituent le cœur du projet. Sans elles, l'application n'est pas démontrable et les compétences ciblées ne sont pas couvertes.

US01 [MVP] — Inscription

En tant que visiteur non inscrit, **je veux** créer un compte, **afin de** pouvoir participer à des soirées.

Critères d'acceptation

- Le formulaire demande au minimum : un email, un pseudo, un mot de passe (saisi deux fois pour confirmation)
- Une confirmation visuelle s'affiche après inscription réussie
- En cas d'erreur (email déjà pris, format invalide...), un message clair indique le problème
- Après inscription, l'utilisateur est connecté ou redirigé vers la page de connexion

Contraintes

- L'email doit être unique sur l'ensemble des comptes
- Le pseudo doit être unique sur l'ensemble des comptes
- Le mot de passe fait au moins 8 caractères
- Le mot de passe n'est **jamais** stocké en clair
- Un nouveau compte a par défaut le rôle "membre"
- La date de création du compte est conservée

US02 [MVP] — Connexion

En tant que utilisateur inscrit, **je veux** me connecter, **afin de** accéder aux fonctionnalités réservées aux membres.

Critères d'acceptation

- Connexion via email + mot de passe
- En cas d'erreur, un message **générique** s'affiche (ne pas révéler si l'erreur vient de l'email ou du mot de passe)
- Après connexion, redirection vers la page d'accueil membre

Contraintes

- L'authentification utilise un mécanisme sécurisé (JWT)

US04 [MVP] — Consulter le catalogue de jeux

En tant que membre, **je veux** consulter le catalogue de jeux, **afin de** connaître les jeux disponibles pour les soirées.

Critères d'acceptation

- Liste paginée ou non, affichant pour chaque jeu : titre, nombre de joueurs (min - max), durée moyenne, complexité, image (si disponible)
- Au clic sur un jeu : description complète
- Filtres disponibles : par nombre de joueurs et par durée
- Tri possible par titre

Contraintes

- Tous les membres connectés peuvent consulter

US05 [MVP] — Gérer le catalogue de jeux

En tant que admin, **je veux** ajouter, modifier et supprimer des jeux du catalogue, **afin de** maintenir l'inventaire à jour.

Critères d'acceptation

- Formulaire de création avec : titre, description, nombre minimum de joueurs, nombre maximum de joueurs, durée moyenne (en minutes), complexité (1 à 5), URL d'image (optionnelle)
- Édition d'un jeu existant via le même formulaire
- Suppression possible avec confirmation

Contraintes

- Action réservée aux admins (un membre normal qui appelle l'API doit recevoir une erreur 403)
- **nombre minimum de joueurs ≥ 1**
- **nombre maximum de joueurs $\geq$ nombre minimum de joueurs**
- Complexité comprise entre 1 et 5
- Durée strictement positive
- Le titre doit être unique dans le catalogue

US06 [MVP] — Créer une soirée

En tant que membre, **je veux** créer une soirée, ****afin d'**inviter d'autres membres à venir jouer.**

Critères d'acceptation

- Formulaire avec : titre, date et heure, lieu (texte libre), capacité maximum, sélection d'un ou plusieurs jeux du catalogue
- Confirmation visuelle après création
- Redirection vers la page de détail de la soirée nouvellement créée

Contraintes

- La date et l'heure doivent être strictement dans le futur
- La capacité doit être ≥ 2
- Au moins un jeu doit être sélectionné
- Tous les jeux sélectionnés doivent être compatibles avec la capacité (le **nombre minimum de joueurs** du jeu doit être $\leq$ capacité de la soirée)
- L'utilisateur qui crée la soirée en devient automatiquement l'hôte
- L'hôte est automatiquement enregistré comme participant confirmé (RSVP "oui")

US07 [MVP] — Lister les soirées

En tant que membre, **je veux** voir la liste des soirées, **afin de** choisir lesquelles rejoindre ou consulter.

Critères d'acceptation

- Deux sections distinctes : soirées **à venir** et soirées **passées**
- Pour chaque soirée affichée : titre, date, lieu, pseudo de l'hôte, nombre de participants confirmés / capacité
- Filtre supplémentaire : "mes soirées" (celles que j'organise + celles auxquelles je participe)

US08 [MVP] — Voir le détail d'une soirée

En tant que membre, **je veux** consulter le détail d'une soirée, **afin de** décider si je participe et de voir ce qui s'y passe.

Critères d'acceptation

- Affichage complet : titre, date, lieu, hôte, capacité, jeux apportés, liste des participants confirmés (par pseudo)
- Mon statut RSVP s'il est défini
- Bouton pour définir / modifier ma RSVP (si soirée à venir)
- Pour une soirée passée à laquelle j'ai participé : section permettant de noter les jeux (cf. US11)

Contraintes

- L'email des autres participants ne doit jamais être exposé

US09 [MVP] — Indiquer ma présence (RSVP)

En tant que membre, **je veux** indiquer ma présence à une soirée, **afin que** l'hôte sache à quoi s'attendre.

Critères d'acceptation

- Trois choix possibles : "Je viens", "Je ne viens pas", "Peut-être"
- Possibilité de modifier ma réponse tant que la soirée n'est pas passée
- La date de la dernière réponse est conservée

Contraintes

- Un utilisateur ne peut avoir qu'une seule réponse active par soirée (pas d'historique multiple)
- Impossible de RSVP à une soirée passée
- Le nombre de RSVP "oui" ne peut **pas** dépasser la capacité de la soirée — toute tentative au-delà doit être refusée avec un message clair (pas de file d'attente)
- L'hôte ne peut pas se mettre RSVP "non" ni "peut-être" à sa propre soirée

US11 [MVP] — Noter les jeux d'une soirée passée

En tant que participant confirmé, **je veux** noter les jeux joués lors d'une soirée passée, **afin de** garder une trace de ce qui m'a plu et alimenter une mémoire collective.

Critères d'acceptation

- Pour chaque jeu apporté à la soirée, un formulaire propose : note de 1 à 5 (obligatoire) + commentaire (optionnel, texte libre)
- Une fois ma note enregistrée, je peux la modifier
- Les notes (et leurs auteurs) sont visibles par tous les participants confirmés de la soirée
- Affichage de la note moyenne par jeu et par soirée

Contraintes

- Je dois avoir été participant confirmé (RSVP "oui") à la soirée concernée
- La soirée doit être passée (date et heure < maintenant)
- Je ne peux noter qu'un jeu effectivement présent à la soirée
- Je ne peux noter qu'**une seule fois** un jeu donné pour une soirée donnée (mais je peux modifier cette note)
- La note doit être un entier entre 1 et 5
- La date de notation est conservée

Extensions — À traiter si le MVP est solide

Ces user stories enrichissent l'application sans en être indispensables. **Ne les abordez qu'après avoir verrouillé tout le MVP** (US01-US11), tests et industrialisation comprises. Une extension partiellement implémentée fait perdre des points : ne commencez que celles que vous pouvez **finir proprement**.

US03 [Extension] — Consulter et modifier mon profil

En tant que membre connecté, **je veux** consulter et modifier mon profil, **afin de** tenir mes informations à jour.

Critères d'acceptation

- La page profil affiche : pseudo, email, avatar (si renseigné), date de création du compte
- Possibilité de modifier : pseudo, email, URL d'avatar, mot de passe
- Le mot de passe actuel est exigé pour changer le mot de passe
- Confirmation visuelle après chaque modification

Contraintes

- Email et pseudo restent uniques après modification
- L'avatar est représenté par une URL (pas d'upload de fichier)

US10 [Extension] — Annuler une soirée

En tant que hôte, **je veux** annuler ma soirée, **afin de** prévenir les participants quand elle ne peut pas avoir lieu.

Critères d'acceptation

- Bouton "Annuler" sur la page de détail de la soirée
- Confirmation requise avant action
- La soirée passe dans un état "annulée"
- Les participations existantes sont conservées (utiles pour l'historique)
- Une soirée annulée n'apparaît plus dans la liste "à venir"

Contraintes

- Seul l'hôte ou un admin peut annuler
- Une soirée annulée ne peut pas être réactivée
- Impossible de RSVP à une soirée annulée
- Cette US implique d'enrichir le modèle de données (**champ statut sur la soirée** ou équivalent) — anticipez-la dans le MCD si vous comptez la faire, ou prévoyez une migration sinon

US12 [Extension] — Consulter mon historique

En tant que membre, **je veux** consulter mon historique, **afin de** me rappeler les soirées vécues et les jeux joués.

Critères d'acceptation

- Liste des soirées passées auxquelles j'ai participé (RSVP "oui")
- Pour chaque soirée : titre, date, hôte, jeux joués avec mes notes
- Statistiques personnelles simples : nombre total de soirées, nombre de jeux différents joués, ma note moyenne donnée

Contraintes

- Si je n'ai jamais participé à rien, un message d'état vide s'affiche
- Les soirées annulées (si l'US10 est implémentée) n'apparaissent pas dans l'historique

Modération admin — Bonus si tout le reste est complet

Ces user stories n'apportent pas de nouvelle compétence évaluée — le rôle admin est déjà exercé via l'US05. Elles ne valent la peine que si l'application complète est livrée et solide.

US13 [Modération] — Désactiver un compte utilisateur

En tant que admin, **je veux** désactiver un compte, **afin de** modérer la plateforme en cas d'abus.

Critères d'acceptation

- Liste des utilisateurs avec recherche par pseudo
- Bouton "désactiver" / "réactiver" selon l'état actuel
- Un compte désactivé ne peut plus se connecter
- Les données associées (soirées organisées, RSVP, notes) sont conservées

Contraintes

- Un admin ne peut pas se désactiver lui-même
- Action réservée aux admins
- Cette US implique d'enrichir le modèle utilisateur (**indicateur d'activation** ou équivalent)

US14 [Modération] — Supprimer une soirée problématique

En tant que admin, **je veux** supprimer une soirée, **afin de** modérer la plateforme.

Critères d'acceptation

- Bouton de suppression accessible depuis la page de détail
- Confirmation requise
- La suppression entraîne la suppression des données associées (RSVP, notes liées)

Contraintes

- Action réservée aux admins
- L'opération est irréversible

6. Règles métier transverses

Ces règles complètent les contraintes inscrites dans les user stories. Certaines recoupent volontairement les US — c'est normal, elles servent de checklist pour la modélisation et les tests. Les règles marquées Extension ou Modération ne s'appliquent que si l'extension correspondante est implémentée.

6.1 Cohérence temporelle

- Toutes les actions doivent être horodatées (création de compte, RSVP, notation, etc.)

- La notion de “soirée passée” se base sur **date et heure de la soirée < instant courant**
- Une soirée ne peut pas être créée dans le passé ni dans le présent immédiat

6.2 Sécurité et accès

- Toute action en écriture est soumise à authentification
- Les actions admin sont protégées par contrôle de rôle côté serveur (jamais uniquement côté frontend)
- Un utilisateur désactivé ne peut effectuer aucune action authentifiée (US13)

6.3 Intégrité des données

- L'unicité de l'email et du pseudo est garantie au niveau base de données
- Une RSVP est unique par couple (utilisateur, soirée)
- Une note est unique par triplet (utilisateur, jeu, soirée)
- La présence d'un jeu à une soirée est unique par couple (jeu, soirée)
- La suppression d'un jeu du catalogue ne doit pas casser les soirées passées qui le mentionnent (réflexion attendue : suppression logique ? cascade ? interdiction si historique ? — argumentez votre choix)

6.4 Validations à appliquer côté serveur

- Tous les champs textes obligatoires doivent être non vides après trim
- Les longueurs maximales raisonnables sont à fixer et documenter (titres, descriptions, commentaires...)
- Les entrées numériques doivent être validées dans leurs bornes (complexité 1-5, note 1-5, etc.)
- Toute violation de contrainte renvoie un code HTTP approprié (400, 403, 404, 409 selon le cas) avec un message explicite

Astuce conception : avant de coder une seule ligne, listez sur papier toutes les **entités** que vous identifiez dans le brief, leurs **attributs**, et les **relations** entre elles avec leurs cardinalités. Concentrez-vous d'abord sur le périmètre **MVP** ; n'incluez les attributs propres aux extensions que si vous comptez vraiment les livrer. À défaut, prévoyez une migration ultérieure.

7. Exigences techniques

7.1 Architecture

- Application **multicouche** : présentation / métier / persistance clairement séparées côté backend
- API **REST** documentée (OpenAPI / Swagger)
- Frontend **SPA** consommant l'API
- Persistance dans une **base relationnelle**

7.2 Sécurité

- Authentification par **JWT** ou **session sécurisée**
- Mots de passe **hashés** (bcrypt ou argon2 — jamais en clair, jamais MD5/SHA1 nu)
- Protection contre les failles **OWASP Top 10** classiques : injection SQL, XSS, CSRF (selon archi)
- Validation **systématique** des entrées côté serveur
- Politique CORS explicite
- Aucun secret en dur dans le code → variables d'environnement + fichier **.env.example** versionné, vrai **.env** dans le **.gitignore**

7.3 Tests

- **Tests unitaires** sur la couche service / logique métier
- **Tests fonctionnels** sur les endpoints critiques (auth, règles métier sensibles)
- **Fixtures** de test reproductibles (seed de données cohérente avec un script lançable en une commande)

7.4 Qualité de code

- **Linter** configuré et passant
- **Conventions de commit** type [Conventional Commits](#)
- Workflow Git lisible : branches de feature, merges vers **main** ou **develop**
- Pas de fichier inutile commité (**node_modules**, **.env**, dumps, captures)

7.5 Industrialisation

- **Dockerfile** pour le backend, **Dockerfile** pour le frontend
- **docker-compose.yml** orchestrant : frontend + backend + base de données (+ reverse proxy en bonus)
- L'application doit se lancer entièrement avec **docker compose up**, sans intervention manuelle après cela
- **Pipeline CI/CD** (GitHub Actions ou GitLab CI) qui :
 - Lance le linter
 - Lance les tests
 - Construit les images Docker
- **Documentation** :
 - **README.md** complet (cf. section 10)
 - Documentation API auto-générée (Swagger UI accessible)
 - Schéma d'architecture simple
 - **MCD et MLD** documentés dans **docs/** avec justification des choix de modélisation

8. Stack technique recommandée

Choisissez librement parmi ces combinaisons éprouvées. Privilégiez la stack que vous **maîtrisez le mieux** — la semaine est courte et n'est pas le bon moment pour découvrir un framework.

8.1 Backend

Stack	Variante typique
Node.js	Express ou NestJS, Prisma ou TypeORM
PHP	Symfony, Doctrine
Python	FastAPI ou Django REST, SQLAlchemy ou Django ORM
Java	Spring Boot, Spring Data JPA
C#	ASP.NET Core, Entity Framework

8.2 Frontend

Stack	Notes
React + Vite	Avec React Router et Axios ou Tanstack Query
Vue 3 + Vite	Avec Vue Router et Pinia
Angular	Si vous le maîtrisez déjà

8.3 Base de données

PostgreSQL recommandé. MySQL/MariaDB acceptés. SQLite envisageable en dev pour aller plus vite, mais PostgreSQL/MySQL en docker-compose pour la livraison finale.

8.4 Outils transverses

- **Versioning** : Git + GitHub ou GitLab
- **CI/CD** : GitHub Actions ou GitLab CI
- **Conteneurisation** : Docker, docker-compose
- **Maquettes** : Figma, Penpot ou Excalidraw
- **UML & MCD** : [draw.io](#), PlantUML, Mocodo, ou Mermaid
- **Doc API** : Swagger UI ou Redoc
- **Test API manuel** : Postman, Insomnia, ou Bruno

9. Planning indicatif (5 jours)

Le planning ci-dessous est un fil rouge **focalisé sur le MVP**. Les extensions et la modération ne sont mentionnées qu'en option. Si vous prenez du retard, **réduisez en restant sur le MVP**, ne tentez pas d'extensions.

Jour 1 — Conception (MVP uniquement)

Matin

- Relecture du brief, questions éventuelles
- User stories MVP priorisées et comprises
- Choix de la stack et justification écrite
- **Identification des entités, attributs, relations et cardinalités** à partir des US MVP et règles métier
- Premier jet de **MCD** centré sur le MVP, validation avant de poursuivre

Après-midi

- **MLD** ou diagramme de classe dérivé du MCD, avec contraintes d'intégrité et types
- Diagrammes UML : cas d'utilisation, **un** diagramme de séquence (par exemple sur le flow "noter un jeu après une soirée")
- Wireframes des écrans MVP (5 écrans — cf. 9.bis)
- Initialisation du repo Git, structure des dossiers, **.gitignore**, README minimal, linter configuré

Livrable de fin de journée : tous les documents de conception poussés dans un dossier **docs/**, MCD/MLD validés par moi.

Jour 2 — Backend socle (MVP)

- Initialisation du projet backend
- Configuration de la connexion à la base, premières migrations basées sur le MLD
- Modèles / entités correspondant à votre modélisation
- Authentification (inscription, login, hash, génération de token) → US01, US02
- Middleware de protection des routes + gestion du rôle admin
- Endpoint **GET /me**
- CRUD basique sur les jeux et les soirées → US04, US05, US06
- Premiers tests unitaires sur le service d'authentification
- Configuration de Swagger / OpenAPI

Livrable de fin de journée : auth fonctionnelle testable via Postman/curl, CRUD de base.

Jour 3 — Backend métier & tests (MVP)

- Endpoints pour : RSVP, notation, sélection des jeux apportés → US07, US08, US09, US11
- Implémentation de toutes les **règles métier MVP** des sections 5 et 6
- Fixtures de test cohérentes (3 utilisateurs dont 1 admin, 6 jeux, 3 soirées à différents stades : future et passées)
- Tests unitaires sur les services métier
- Tests fonctionnels sur les endpoints sensibles
- Vérification de la couverture de tests
- Doc Swagger à jour

Livrable de fin de journée : backend MVP complet, tests verts, couverture ≥ 50%, doc API navigable.

Point de vérification : si à la fin du jour 3 le backend MVP n'est pas terminé et testé, **n'envisagez aucune extension**. Concentrez-vous sur le frontend du MVP au jour 4.

Jour 4 — Frontend (MVP)

- Initialisation du projet frontend
- Routing et structure des pages
- 5 écrans MVP (cf. 9.bis)
- Gestion du token (stockage, ajout dans les headers, déconnexion)
- Gestion conditionnelle de l'UI selon le rôle
- Au moins 2-3 composants réutilisables
- Au moins un test composant
- Responsive desktop + mobile

Livrable de fin de journée : frontend MVP fonctionnel branché sur le backend, parcours utilisateur complet jouable de bout en bout.

Jour 5 — Industrialisation & livraison

Matin

- Dockerfile backend, Dockerfile frontend
- docker-compose orchestrant tout (front, back, db)
- Vérification : **docker compose up** lance toute l'application sans intervention manuelle
- Pipeline CI/CD : lint → tests → build des images
- Polissage : messages d'erreur, états de chargement, edge cases UX

Après-midi

- Rédaction finale du README
- Documentation d'architecture (schéma + texte)
- Simulation de déploiement (staging local via compose, ou déploiement réel sur un PaaS gratuit selon ambition)

Livrable de fin de journée : projet complet, repo propre, soutenance prête.

Si à la fin du jour 4 vous êtes en avance : vous pouvez tenter une extension (US03, US10, ou US12) en début de jour 5, **avant** la dockerisation. Choisissez **une seule** US, finissez-la complètement, puis revenez au planning.

9.bis — Liste des écrans frontend MVP (5 écrans)

1. **Login / Register** (deux formulaires sur une page ou deux pages liées)
2. **Liste des soirées** (à venir et passées, filtrables)
3. **Détail d'une soirée** (infos, jeux apportés, RSVP, notation pour les soirées passées)
4. **Création d'une soirée** (formulaire avec sélection multi-jeux)
5. **Catalogue de jeux** (consultation et, pour les admins, gestion)

Le profil et l'historique (extensions) ajouteraient un 6e écran si traités.

10. Livrables attendus

À la fin de la semaine, vous devez livrer :

10.1 Repository Git

- Structure claire et documentée (séparation **backend/**, **frontend/**, **docs/**)
- Branche **main** avec un historique de commits propre
- Tag ou release pour la version finale (**v1.0.0**)

10.2 [README.md](#) à la racine

Le README doit contenir au minimum :

- Titre et description du projet
- **Périmètre livré** (lister les US livrées avec leur niveau MVP / Extension / Modération)
- Stack technique utilisée
- Prérequis (versions de Node, PHP, Docker, etc.)
- Instructions d'installation pas à pas
- Instructions de lancement (mode dev et mode docker)
- Instructions pour lancer les tests
- Comptes de démo (admin + membre) avec mots de passe en clair pour la correction
- Lien vers la documentation API
- Description rapide des choix techniques

10.3 Documentation technique (**docs/**)

- **MCD et MLD** avec une note expliquant les choix de modélisation (notamment la gestion des suppressions, les choix d'unicité, les éventuelles dénormalisations)
- Diagrammes UML (cas d'usage, classes, séquence)
- Maquettes (PNG ou lien Figma)
- Schéma d'architecture

- Document court sur les mesures de sécurité prises

10.4 Application opérationnelle

- Backend testé (couverture mesurable et affichée)
- Frontend fonctionnel
- `docker compose up` lance toute l'application
- Documentation API (Swagger UI accessible à une URL connue)
- Pipeline CI verte sur le dernier commit de `main`

11. Bonus si vous avez fini en avance

Ordre d'attaque conseillé :

1. **D'abord** : verrouiller toutes les US MVP, tests inclus, couverture $\geq 60\%$
2. **Ensuite** : dockerisation et CI complètes
3. **Puis** : extensions (US03, US10, US12) — une par une, finies proprement
4. **Enfin** : modération admin (US13, US14)

Si toutes les US sont livrées et qu'il vous reste du temps :

- Augmenter la couverture de tests à 80%+
- Ajouter un reverse proxy (nginx ou traefik) au docker-compose
- Ajouter un test end-to-end avec Cypress ou Playwright sur le parcours principal
- Pagination + recherche avancée sur le catalogue de jeux
- Page admin avec un dashboard de stats simples
- Déploiement réel sur un PaaS gratuit (Render, [Fly.io](#), Railway)
- Mode sombre côté frontend
- Export de l'historique personnel en CSV ou PDF

Aucun bonus ne compense un MVP incomplet ou cassé. **Finir solidement** prime sur **ajouter en surface**.

12. Conseils & pièges à éviter

12.1 Conseils pour tenir une semaine seul

- **Time-boxez chaque demi-journée**. Si vous n'avez pas terminé ce qui était prévu, coupez au lieu d'allonger.
- **Commencez chaque journée par 10 minutes de planification écrite** : qu'est-ce que je veux avoir terminé ce soir ?
- **Faites valider votre MCD avant le jour 2**. Une erreur de modélisation détectée le jour 4 coûte une demi-journée de refactor.
- **Décidez le jour 1 si vous tenterez les extensions** — cela conditionne votre MCD. Dans le doute, modélisez MVP-only et migrez plus tard si besoin.
- **Commits petits et fréquents**, message clair. Pas de commit "wip lol" sur `main`.
- **Gardez `main` déployable en permanence**. À tout moment, `docker compose up` doit marcher.
- **Écrivez le README au fur et à mesure**, pas le vendredi à 17h.
- **N'attendez pas le dernier jour pour Docker**. Créez un Dockerfile dès le jour 2, ne serait-ce que minimal.
- **Si vous séchez plus de 30 minutes** sur un point, n'hésitez pas à me solliciter.

12.2 Pièges classiques

- Vouloir refaire BoardGameGeek
- Tenter une extension sans avoir verrouillé le MVP
- Empiler les frameworks ou les libs sans en maîtriser un seul
- Bâcler le MCD pour "se mettre à coder vite" — vous le paierez plus tard
- Coder les règles métier dans les contrôleurs au lieu d'une couche service
- Oublier les fixtures et tester sur des données entrées à la main
- Hardcoder l'URL de l'API ou le secret JWT
- Pousser le `.env` réel sur Git (vérifiez votre `.gitignore` dès le premier commit)

- Faire les tests "à la fin", il n'y a jamais "la fin"
- Faire un docker-compose qui marche "presque" mais demande 3 manipulations manuelles

12.3 Pour le dossier professionnel

Cette mission couvre largement les compétences évaluées en CCP1, CCP2 et CCP3. Au fil de la semaine, **prenez des notes** sur :

- Les choix techniques que vous faites et pourquoi
- Les difficultés rencontrées et comment vous les avez résolues
- Les compromis que vous avez dû faire (notamment : qu'avez-vous laissé en backlog et pourquoi ?)

Ces notes seront précieuses pour rédiger le DP.